

Deploying Cloud-Native Apps on GCP with Antigravity CLI



S.E.N.S.E

Signal Event and Notification Service Engine

Alex Nyambura
Cloud DevOps Engineer | Spider-Man



Operational Constraints of a Solo Hero

THE CONTEXT

The Peter Parker Situation

After the world forgot him, Peter lost all Stark tech, AI, and funding. He is broke and operating from a cramped apartment.

The Challenge: Citizens need to send panic signals to a visual HUD map in real-time.

Engineering Constraint: It MUST cost \$0 when inactive (scale-to-zero) but handle thousands of requests instantly during a Sinister Six attack.



HUD Visualization: Real-time emergency tracking

Defining **Cloud-Native**: Modular Efficiency

The Core Philosophy

Designing applications to fully leverage cloud infrastructure through modularity, replacing rigid "building-integrated" kitchens with adaptable containers.

Architectural Pillars

- **Easy to Scale:** Add capacity instantly as demand grows.
- **Fault Tolerant:** Swap broken modules without downtime.
- **Frequently Deployable:** Update components independently.
- **Automated:** Self-managing infrastructure and recovery.
- **Loosely Coupled:** Independent services communicating via APIs.



Why Cloud "Native"?

Embracing the realities of the modern environment

1. Containers & Orchestration

Docker packages apps with dependencies; Kubernetes manages the fleet.

- Restart crashed containers
- Auto-scale on demand
- Zero-downtime rollouts

2. Microservices

Decoupling "One Big App" into independent, resilient services.

- Independent deployment
- Isolated failure domains
- Specific scaling (e.g. Payments)

3. Automation: CI/CD & IaC

Infrastructure is code; changes move to production via pipelines.

- Consistent environments
- Rapid release cycles
- Version-controlled infra

Core Realities

- Servers are ephemeral
- Failures are expected
- Capacity is fluid
- Deployment is continuous



Modernizing with Serverless Google Cloud Services



Google Cloud Run

Scales to zero when idle; spins up in milliseconds for requests.



Google Cloud Pub/Sub

Guarantees delivery and buffers massive traffic spikes safely.



Cloud Firestore

NoSQL with real-time sync; pushes updates without polling.

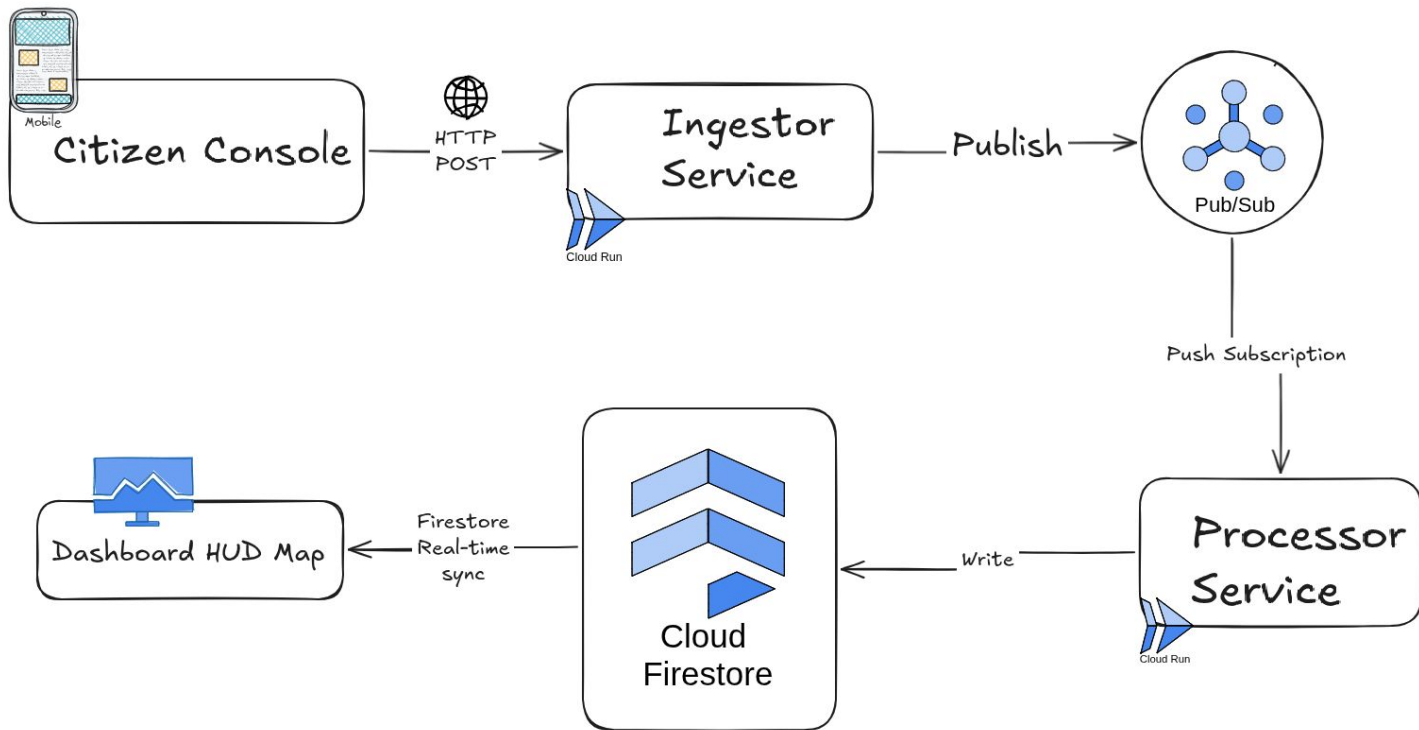


Citizen Distress Call Architecture Trace

- 01 Citizen Console → Ingestor**
HTTP POST request to Cloud Run service for immediate validation.
- 02 Ingestor → Pub/Sub**
Asynchronous publishing to Topic; returns 200 OK to citizen instantly.
- 03 Pub/Sub → Processor**
Push Subscription triggers Cloud Run instance to parse data.
- 04 Firestore → Dashboard**
Coordinates written to Firestore; real-time sync updates HUD Map.



S.E.N.S.E Architecture



Technical vs. Non-Technical Service Breakdown

01

Ingestor Service

Analogy: Spidey's dispatcher. Takes the call, notes location, drops in mailbox, moves to next call.

Python FastAPI. Minimal overhead—serializes and publishes to Pub/Sub.

02

Google Pub/Sub

Analogy: The post office. Mail sits safely in the mailbox until the postman is ready to sort it.

Message broker for decoupling. Asynchronous processing with automatic retries if offline.

03

Processor Service

Analogy: The archivist. Retrieves letters from the mailbox and logs them into the master ledger.

FastAPI triggered by webhook push. Uses Firestore SDK to write records to NoSQL.

04

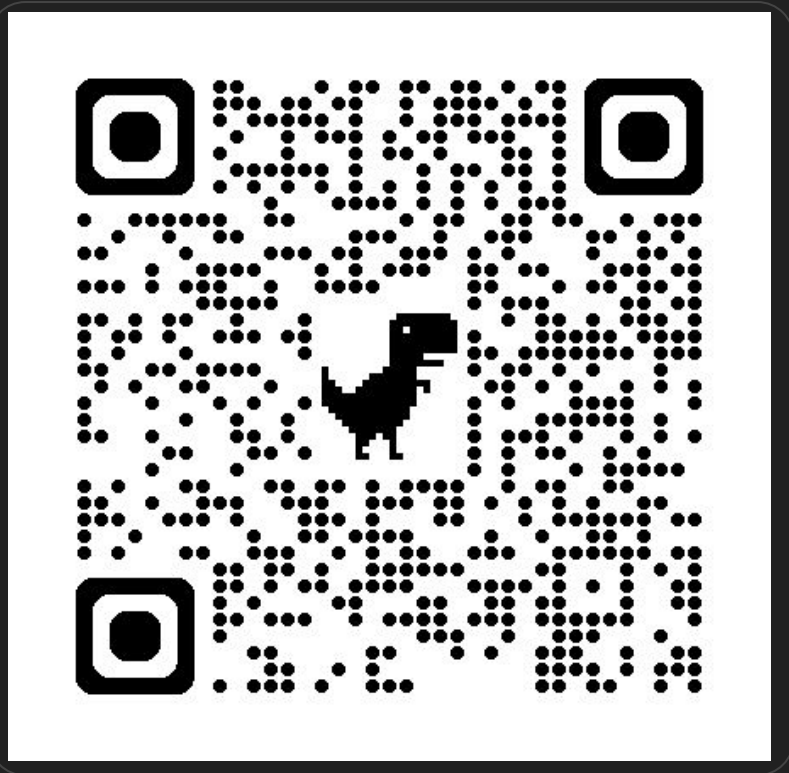
Cloud Firestore & Dashboard HUD

Analogy: Master ledger that automatically updates tactical maps in the Spider-Cave.

Real-time listeners (onSnapshot) push deltas. Frontend renders dynamic, density-scaled beacons.



Code Walkthrough



<https://github.com/lxmwaniky/sense>

**“With great power
comes great
responsibility.”**

Uncle Ben